

QUIS DATA PARALLELISM	Nama: Farhan Kamil Hermansyah NRP: 152024150 Mata Kuliah: IFB 206 - Komputasi Paralel dan Sistem Terdistribusi Dosen Pengampu: Lisa Kristiana PhD
------------------------------	--

Github :

https://github.com/Farmil23/IFB-206-KOMPUTASI-PARALEL-SISTEM-TERDISTRIBUSI/tree/main/LECTURE_7_QUIS

1. DATA PARRALELISM

SOURCE KODE:

```
import os
import time
from multiprocessing import Pool
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv

load_dotenv()

# =====
# 1. SETUP: Membuat "Different Data"
# =====

def create_rag_documents():
    """Membuat 4 dokumen konteks yang berbeda (Different Data)"""
    docs = {
        "export_ai_compliance.txt": "Regulasi ekspor UMKM memerlukan validasi dokumen kepatuhan secara ketat menggunakan vision extraction dan penalaran terstruktur. ",
        "scholarsync_journals.txt": "Pengindeksan dokumen saintifik dalam format PDF dapat dianalisis secara semantik untuk membantu asisten peneliti melakukan tinjauan pustaka otonom. ",
        "graphweaver_docs.txt": "Investigasi forensik digital memanfaatkan autonomous knowledge graph builder yang didukung oleh database graf dan multi-agent workflow. ",
        "farmile_portfolio.txt": "Portofolio pengembangan sistem automasi pertanian mencakup berbagai integrasi sensor dan pemetaan area operasional untuk efisiensi lahan. "
    }

    file_paths = []

    for filename, content in docs.items():
        # Memperbanyak teks agar chunking process lebih realistis
        with open(filename, "w", encoding="utf-8") as f:
            f.write((content + "\n") * 100)
        file_paths.append(filename)

    return file_paths
```

```

# =====
# 2. THE SAME TASK (Tugas Paralel)
# =====

def process_single_document(file_path):
    """
    TUGAS YANG SAMA: Setiap prosesor memuat satu dokumen dan memecahnya
    menjadi chunks.

    Ini adalah CPU-bound task yang sangat cocok untuk Data Parallelism.
    """
    process_id = os.getpid()
    print(f"[Worker PID: {process_id}] Memulai ingestion data pada: {file_path}")

    # Load dan Split
    loader = TextLoader(file_path, encoding="utf-8")
    document = loader.load()

    text_splitter = RecursiveCharacterTextSplitter(chunk_size=300, chunk_overlap=80)
    chunks = text_splitter.split_documents(document)

    time.sleep(1) # Simulasi komputasi berat agar paralelisasi terlihat di log
    print(f"[Worker PID: {process_id}] Selesai memproses {file_path} -> {len(chunks)}
    chunks.")
    return chunks

# =====
# 3. MAIN EXECUTION (Data Parallelism)
# =====

def parallel_retrieve_docs():
    file_paths = create_rag_documents()
    print("=== MEMULAI PARALLEL DOCUMENT INGESTION ===")
    start_time = time.time()

```

```

# Menjalankan Data Parallelism dengan 4 CPU cores
with Pool(processes=4) as pool:
    # pool.map mendistribusikan Task yang SAMA ke Data yang BERBEDA
    parallel_results = pool.map(process_single_document, file_paths)

# Menggabungkan hasil dari semua worker menjadi satu list (flatten list)
all_chunks = [chunk for sublist in parallel_results for chunk in sublist]
end_time = time.time()

print(f"\nTotal waktu eksekusi paralel: {end_time - start_time:.4f} detik")
print(f"Total chunks dari semua dokumen: {len(all_chunks)}")
print("Membangun FAISS Vectorstore...\n")

# Embeddings (Memasukkan semua chunk ke dalam FAISS)
embedding = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(documents=all_chunks,
embedding=embedding)

# Bersihkan file sementara
for file in file_paths:
    if os.path.exists(file):
        os.remove(file)

return vectorstore.as_retriever()

def sequential_retrieve_docs():
    file_paths = create_rag_documents()
    print("=== MEMULAI PARALLEL DOCUMENT INGESTION ===")
    start_time = time.time()

# Menjalankan Data Parallelism dengan 4 CPU cores
with Pool(processes=1) as pool:
    # pool.map mendistribusikan Task yang SAMA ke Data yang BERBEDA
    parallel_results = pool.map(process_single_document, file_paths)

```

```

# Menggabungkan hasil dari semua worker menjadi satu list (flatten list)
all_chunks = [chunk for sublist in parallel_results for chunk in sublist]
end_time = time.time()

print(f"\nTotal waktu eksekusi paralel: {end_time - start_time:.4f} detik")
print(f"Total chunks dari semua dokumen: {len(all_chunks)}")
print("Membangun FAISS Vectorstore...\n")

# Embeddings (Memasukkan semua chunk ke dalam FAISS)
embedding = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(documents=all_chunks,
embedding=embedding)

# Bersihkan file sementara
for file in file_paths:
    if os.path.exists(file):
        os.remove(file)

return vectorstore.as_retriever()
# Eksekusi untuk dites di Notebook
if __name__ == '__main__':
    # Kamu bisa mengganti 'retriever = retrieve_docs()' di cell 30 dengan ini:
    retriever = parallel_retrieve_docs()

```

HASIL EKSEKUSI DATA PARRALELISM:

```
(rag_app) D:\LIFE_KULIAH_ITENAS_SEMESTER 4\KOMPUTASI PARALLEL>python LECTURE_7_QUIS\RAG_STUDY_CASE.py
=== MEMULAI PARALLEL DOCUMENT INGESTION ===
[Worker PID: 24164] Memulai ingestion data pada: export_ai_compliance.txt
[Worker PID: 24436] Memulai ingestion data pada: scholarsync_journals.txt
[Worker PID: 23932] Memulai ingestion data pada: graphweaver_docs.txt
[Worker PID: 24416] Memulai ingestion data pada: farmile_portfolio.txt
[Worker PID: 24164] Selesai memproses export_ai_compliance.txt -> 50 chunks.
[Worker PID: 24436] Selesai memproses scholarsync_journals.txt -> 50 chunks.
[Worker PID: 23932] Selesai memproses graphweaver_docs.txt -> 50 chunks.
[Worker PID: 24416] Selesai memproses farmile_portfolio.txt -> 50 chunks.

Total waktu eksekusi Parralel: 7.8806 detik
Total chunks dari semua dokumen: 200
Membangun FAISS Vectorstore...
```

Pada gambar di atas, sistem mengimplementasikan konsep *Data Parallelism* dengan memecah dataset ke dalam 4 *Worker* (inti CPU) yang berbeda secara simultan. Setiap PID menjalankan sebuah tugas (task) yang identik—yaitu memuat dokumen teks dan memecahnya menjadi *chunks*—namun dengan **asupan data yang berbeda**. Berikut adalah rincian tugas yang dieksekusi secara serentak:

- **Worker PID 24164:** Menjalankan tugas memuat dan mengekstraksi data dari file `export_ai_compliance.txt` secara mandiri hingga menghasilkan 50 *chunks*.
- **Worker PID 24436:** Menjalankan tugas memuat dan mengekstraksi data dari file `scholarsync_journals.txt` secara mandiri hingga menghasilkan 50 *chunks*.
- **Worker PID 23932:** Menjalankan tugas memuat dan mengekstraksi data dari file `graphweaver_docs.txt` secara mandiri hingga menghasilkan 50 *chunks*.
- **Worker PID 24416:** Menjalankan tugas memuat dan mengekstraksi data dari file `farmile_portfolio.txt` secara mandiri hingga menghasilkan 50 *chunks*.

Karena keempat *core* tersebut bekerja mengeksekusi instruksinya masing-masing secara paralel, pemrosesan 200 *chunks* dapat diselesaikan lebih efisien dengan total waktu komputasi 7.8806 detik.

Hasil Eksekusi tanpa Data Parralelism:

```
(rag_app) D:\LIFE\KULIAH ITENAS\SEMESTER 4\KOMPUTASI PARALLEL>python LECTURE_7_QUIZ\RAG_STUDY_CASE.py
=== MEMULAI PARALLEL DOCUMENT INGESTION ===
[Worker PID: 13192] Memulai ingestion data pada: export_ai_compliance.txt
[Worker PID: 13192] Selesai memproses export_ai_compliance.txt -> 50 chunks.
[Worker PID: 13192] Memulai ingestion data pada: scholarsync_journals.txt
[Worker PID: 13192] Selesai memproses scholarsync_journals.txt -> 50 chunks.
[Worker PID: 13192] Memulai ingestion data pada: graphweaver_docs.txt
[Worker PID: 13192] Selesai memproses graphweaver_docs.txt -> 50 chunks.
[Worker PID: 13192] Memulai ingestion data pada: farmile_portfolio.txt
[Worker PID: 13192] Selesai memproses farmile_portfolio.txt -> 50 chunks.

Total waktu eksekusi Sekuensial: 9.1191 detik
Total chunks dari semua dokumen: 200
Membangun FAISS Vectorstore...
```

Berbeda dengan eksekusi sebelumnya, pada *output* komputasi sekuensial ini, instruksi tidak didistribusikan. Beban kerja dibebankan sepenuhnya pada satu inti prosesor tunggal, yang diidentifikasi dengan **Worker PID 13192**.

Karena bekerja sendirian, **Worker PID 13192** harus mengeksekusi instruksi pemuatan data secara mengantre; mulai dari mengekstraksi `export_ai_compliance.txt`, lalu dilanjutkan dengan `scholarsync_journals.txt`, `graphweaver_docs.txt`, hingga file terakhir `farmile_portfolio.txt`. Karena tidak ada pembagian tugas secara paralel, waktu eksekusi untuk menyelesaikan 200 *chunks* memakan waktu yang lebih lama, yaitu 9.1191 detik.

II. KESIMPULAN

Kesimpulan akhir adalah Komputer modern memiliki prosesor (CPU) dengan sistem *multi-core* (banyak inti). Pada pemrosesan sekuensial, sebuah program hanya berjalan pada 1 *core*, membiarkan *core* lain menganggur (*idle*), sehingga seluruh beban data harus diproses secara antrean (satu per satu).

Sebaliknya, **Data Parallelism** memecah kumpulan data besar menjadi potongan-potongan kecil (*chunks*) dan mendistribusikannya ke seluruh *core* CPU yang tersedia. Karena setiap *core* mengeksekusi instruksi yang sama secara independen dan simultan (bersamaan) pada datanya masing-masing, beban kerja terbagi rata. Hal ini membuat total waktu komputasi menurun secara drastis, berbanding lurus dengan jumlah *core* yang dialokasikan, dikurangi sedikit waktu *overhead* untuk mendistribusikan dan menggabungkan hasil data tersebut.